



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Discover. Learn. Empower.

WORKSHEET 8

Student Name: Piyush Patel

UID: 23BCS12673

Branch: CSE(3rd Year)

Section/Group: Krg-1-A

Semester: 5th

Date of Performance: 09/10/25

Subject Name: ADBMS

Subject Code: 23CSP-333

1. AIM:

Design a robust PostgreSQL transaction system for the students table where multiple student records are inserted in a single transaction.

If any insert fails due to invalid data, only that insert should be rolled back while preserving the previous successful inserts using savepoints.

The system should provide clear messages for both successful and failed insertions, ensuring data integrity and controlled error handling.

2. Tools Used : PostGres

3. Solutions:

```
CREATE TABLE students (  
    id SERIAL PRIMARY KEY,  
    name VARCHAR(50),  
    age INT,  
    class INT  
);  
DO $$  
BEGIN  
    BEGIN  
        INSERT INTO students(name, age, class) VALUES ('Piyush',17,12);  
        INSERT INTO students(name, age, class) VALUES ('Niraj',17,11);  
        INSERT INTO students(name, age, class) VALUES ('Abhi',15,9);  
  
        RAISE NOTICE 'Transaction Successfully Done';  
  
    EXCEPTION  
        WHEN OTHERS THEN  
            RAISE NOTICE 'Transaction Failed...! Rolling back changes.';  
            RAISE;  
    END;  
END;  
$$;
```



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Discover. Learn. Empower.

```
SELECT * FROM students;
```

```
----- WRONG DATA TYPE SCENARIO -----  
BEGIN;
```

```
SAVEPOINT sp1;  
INSERT INTO students(name, age, class) VALUES ('Supriya',16,8);
```

```
SAVEPOINT sp2;  
BEGIN  
    INSERT INTO students(name, age, class) VALUES ('Sahil','wrong',10); -- fails  
EXCEPTION WHEN OTHERS THEN  
    RAISE NOTICE 'Failed to insert Sahil, rolling back to savepoint sp2';  
    ROLLBACK TO SAVEPOINT sp2;  
END;
```

```
INSERT INTO students(name, age, class) VALUES ('Sam',17,10);
```

```
COMMIT;
```

4. Output:

```
CREATE TABLE students (  
    id SERIAL PRIMARY KEY,  
    name VARCHAR(50),  
    age INT,  
    class INT  
);  
DO $$  
BEGIN  
    BEGIN  
        INSERT INTO students(name, age, class)  
        VALUES ('Piyush', 17, 12);  
  
        INSERT INTO students(name, age, class)  
        VALUES ('Niraj', 17, 11);  
  
        INSERT INTO students(name, age, class)  
        VALUES ('Abhi', 15, 9);  
  
        RAISE NOTICE 'Transaction Successfully Done';  
    EXCEPTION  
        WHEN OTHERS THEN  
            RAISE NOTICE 'Transaction Failed..! Rolling back changes.';  
            RAISE;  
    END;  
END;  
$$;  
SELECT * FROM students;
```

STDIN

Input for the program (Optional)

Output:

```
CREATE TABLE
```

```
DO
```

```
id | name | age | class
```

```
-----+-----+-----+-----
```

```
1 | Piyush | 17 | 12
```

```
2 | Niraj | 17 | 11
```

```
3 | Abhi | 15 | 9
```

```
(3 rows)
```

```
psql:commands.sql:26: NOTICE: Transaction Successfully Done
```



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Discover. Learn. Empower.

```
        RAISE NOTICE 'Transaction Successfully Done';
    EXCEPTION
    WHEN OTHERS THEN
        RAISE NOTICE 'Transaction Failed..! Rolling back changes.';
        RAISE;
    END;
END;
$$;
SELECT * FROM students;

BEGIN; -- Start transaction

-- 1st insert (successful)
SAVEPOINT sp1;
INSERT INTO students(name, age, class)
VALUES ('Supriya', 16, 8);

-- 2nd insert (this one will fail)
SAVEPOINT sp2;
BEGIN
    INSERT INTO students(name, age, class)
    VALUES ('Sahil', 'wrong', 10); -- wrong data type
EXCEPTION
    WHEN OTHERS THEN
        RAISE NOTICE 'Failed to insert Sahil, rolling back to savepoint';
        ROLLBACK TO SAVEPOINT sp2;
```

STDIN

Input for the program (Optional)

Output:

CREATE TABLE

DO

id	name	age	class
1	Piyush	17	12
2	Niraj	17	11
3	Abhi	15	9

(3 rows)

BEGIN

SAVEPOINT

INSERT 0 1

SAVEPOINT

psql:commands.sql:26: NOTICE: Transaction Successfully Done

5. Learning Outcomes:

- Understand the concept of PostgreSQL transactions and how to start, commit, and rollback.
- Learn how to use **SAVEPOINT** to handle partial rollbacks within a transaction.
- Practice controlled error handling for individual insert failures without affecting other successful operations.
- Gain experience in maintaining **data integrity** while performing multiple inserts.
- Learn to generate informative **NOTICES** to monitor transaction progress and errors.